

# 腾讯音乐 前端开发（AI产品）

## 一面（技术基础） 1-6

1. 请简单介绍一下你对CSS 盒模型的理解，以及 box-sizing 的区别。
2. 说一下 JS 中的事件循环机制，特别是微任务和宏任务的执行顺序。
3. 在 AI 问答产品中，流式输出通常使用SSE 或者 WebSocket，请对比一下它们的优缺点。
4. 手写一个防抖函数（debounce），并说明它在 AI搜索建议场景下的作用。
5. 说一下 Promise.all 和 Promise.allSettled 的区别，以及在处理多个 AI插件调用时的选型。
6. TypeScript 中的泛型（Generics）有什么作用？请举一个封装 AI 接口请求的例子。

## 二面（深入原理与项目）. 1-8

1. React 的 Fiber 架构解决了什么问题？它是如何实现可中断渲染的？
2. 在开发 AI 聊天界面时，如何处理长列表（历史消息）的性能问题？
3. AI 产品的流式响应（Streaming）如何实现打字机效果，并保证 Markdown 渲染的正确性？
4. 谈谈你对前端工程化中 Monorepo架构的理解，以及在 AI项目中的适用场景。
5. 你们是如何监控AI产品的性能和异常的？特别是针对大模型响应慢的问题。
6. 如何防御针对 AI前端产品的 Prompt Injection（提示词注入）攻击？
7. 介绍一下 Web Worker 的使用场景，它在 AI前端处理中能起到什么作用？
8. 你们项目里是如何做 Webpack 或 Vite 性能优化的？

## 三面（架构与综合能力） 1-5

1. 如果让你从零设计一个 AI Agent的前端交互框架，你会考虑哪些核心模块？
2. 谈谈 RAG（检索增强生成）在前端链路中可以做哪些优化？
3. 在AI产品快速迭代的过程中，如何平衡“业务交付速度”与“代码质量”？
4. 你认为未来1-2年，前端开发在 AI浪潮下会发生哪些本质的变化？
5. 聊聊你在过去一年中遇到的最具有挑战性的技术问题，你是如何解决的？

## 一面（技术基础）

1. 请简单介绍一下你对 CSS 盒模型的理解，以及 box-sizing 的区别。

难度：★

考点：CSS 基础布局原理

**答案要点：**

盒模型是浏览器对 HTML 元素进行布局的基础模型，主要由 content、padding、border、margin 四部分组成。

- 标准盒模型：width 仅包含 content，不包含 padding 和 border。
- IE 盒模型（怪异模式）：width 包含 content、padding 和 border。
- box-sizing 属性：content-box 对应标准模型，border-box 对应 IE 模型。
- 实际应用：AI 产品的对话气泡通常使用 border-box 以方便精确控制宽度。

**常见坑：**

- 混淆 padding 和 margin 对元素实际占用空间的影响。
- 忽略了 box-sizing 的继承性。

**追问：**

- 如果给一个元素设置 padding: 50%，它是相对于谁的宽度计算的？

## 2. 说一下 JS 中的事件循环机制，特别是微任务和宏任务的执行顺序。

**难度：★★**

**考点：**JS 异步执行机制

**答案要点：**

事件循环是 JS 处理异步操作的核心机制，通过任务队列协调同步任务、微任务和宏任务的执行。

- 执行流程：先执行同步代码，然后清空微任务队列，再取出一个宏任务执行，循环往复。
- 微任务：Promise.then、MutationObserver、process.nextTick。
- 宏任务：setTimeout、setInterval、setImmediate、I/O 操作。
- 渲染时机：浏览器通常在微任务队列清空后、下一个宏任务开始前尝试进行 UI 渲染。

**常见坑：**

- 认为 await 之后的代码是同步执行的。
- 错误判断 Promise 构造函数内部代码的执行时机。

**追问：**

- 在 AI 聊天场景中，如果流式输出太快导致页面卡顿，你会如何利用事件循环优化？

## 3. 在 AI 问答产品中，流式输出通常使用 SSE 或者 WebSocket，请对比一下它们的优缺点。

**难度：★★**

**考点：**网络协议与 AI 流式交互

**答案要点：**

SSE 和 WebSocket 是实现 AI 模型流式返回（Stream）的两种主流方案，主要区别在于协议层级和通信方向。

- SSE (Server-Sent Events)：基于 HTTP 协议，单向通信（服务端到客户端），轻量且支持自动重连。
- WebSocket：独立的 TCP 协议，全双工通信，适合需要频繁双向交互的场景。
- 数据格式：SSE 默认是文本流，WebSocket 支持二进制和文本。
- 兼容性：SSE 在现代浏览器支持良好，但受限于 HTTP/1.1 的连接数限制（HTTP/2 可解决）。

#### 常见坑：

- 忽略了 SSE 在 HTTP/1.1 下有 6 个并发连接限制的问题。
- 没考虑到 WebSocket 需要额外的鉴权和心跳检测逻辑。

#### 追问：

- 如果 AI 接口返回的是 Markdown 碎片，前端如何保证渲染不闪烁？

## 4. 手写一个防抖函数 (debounce)，并说明它在 AI 搜索建议场景下的作用。

难度：★★

考点：手写代码能力、函数节流防抖

答案要点：

防抖是指在事件被触发 n 秒后再执行回调，如果在这 n 秒内又被触发，则重新计时。

- 核心逻辑：利用闭包存储 timer，每次触发时先 clearTimeout。
- 应用场景：AI 助手的输入框实时搜索、窗口大小调整。
- 优化点：支持立即执行参数 (leading) 和取消功能 (cancel)。
- 性能意义：减少不必要的 API 请求，降低服务器压力和 Token 消耗。

#### 常见坑：

- 闭包内的 this 指向问题。
- 忘记处理参数传递。

#### 追问：

- 节流 (throttle) 和防抖的区别是什么？

## 5. 说一下 Promise.all 和 Promise.allSettled 的区别，以及在处理多个 AI 插件调用时的选型。

难度：★★

考点：异步并发控制

答案要点：

这两个方法都用于处理多个并发的异步操作，但对失败状态的处理逻辑完全不同。

- Promise.all：只要有一个失败就立即 reject，适用于所有请求必须全部成功的强依赖场景。
- Promise.allSettled：无论成功还是失败都会等待所有 Promise 完成，并返回每个任务的状态。
- 选型建议：在 AI 插件场景中，如果某个插件报错不应影响其他插件展示，应优先使用 allSettled。

- 返回值结构：allSettled 返回一个包含 status 和 value/reason 的对象数组。

**常见坑：**

- 在 Promise.all 中没有对单个 promise 做 catch 处理导致整个链路中断。

**追问：**

- 如果来实现一个带并发限制的 Promise 调度器，你会怎么写？

## 6. TypeScript 中的泛型（Generics）有什么作用？请举一个封装 AI 接口请求的例子。

**难度：★★**

**考点：**TS 类型系统

**答案要点：**

泛型是指在定义函数、接口或类时不预先指定具体类型，而在使用时才指定的特性，增强了代码的复用性和类型安全性。

- 核心价值：在保持类型约束的同时提供灵活性，避免使用 any。
- 约束：可以使用 extends 关键字对泛型进行约束。
- 默认值：可以为泛型指定默认类型。
- 实际应用：封装统一的 AI Response 类型，通过泛型传入具体的业务 Data 类型。

**常见坑：**

- 泛型嵌套过深导致类型推导失效。
- 忽略了泛型约束导致在函数内部无法访问特定属性。

**追问：**

- keyof 和 typeof 关键字在泛型中怎么配合使用？

## 二面（深入原理与项目）

### 1. React 的 Fiber 架构解决了什么问题？它是如何实现可中断渲染的？

**难度：★★★**

**考点：**React 核心原理

**答案要点：**

Fiber 是 React 16 引入的一种新的协调引擎，旨在解决大型应用在更新时产生的掉帧和卡顿问题。

- 核心问题：旧版 Stack Reconciler 递归更新不可中断，长时间占用主线程。
- 解决方式：将更新任务拆分为小片（Fiber 节点），利用 requestIdleCallback 机制在浏览器空闲时执行。
- 优先级调度：通过 Scheduler 给不同任务分配优先级（如用户输入高于数据请求）。

- 双缓存技术：维护 current 树和 workInProgress 树，实现平滑切换。

**常见坑：**

- 误以为 Fiber 提高了计算速度，实际上它只是改变了任务的调度方式。

**追问：**

- 为什么 React 不直接使用 Generator 来实现可中断？

## 2. 在开发 AI 聊天界面时，如何处理长列表（历史消息）的性能问题？

**难度：★★**

**考点：**长列表优化、虚拟滚动

**答案要点：**

AI 聊天场景下，消息量可能非常大且包含复杂的 Markdown 或代码块，直接渲染会导致 DOM 节点过多。

- 虚拟列表：只渲染可视区域内的 DOM 节点，通过 padding 或 transform 模拟滚动条。
- 动态高度处理：AI 回复长度不固定，需要动态计算或缓存每个 Item 的高度。
- 分片渲染：对于非首屏数据，可以采用时间分片的方式逐步插入 DOM。
- 资源回收：及时销毁不可见区域的复杂组件（如代码高亮、公式渲染）。

**常见坑：**

- 滚动过快导致的白屏问题。
- 动态高度计算不准确导致的滚动条跳动。

**追问：**

- 如果消息中包含图片，如何保证虚拟滚动的偏移量计算准确？

## 3. AI 产品的流式响应（Streaming）如何实现打字机效果，并保证 Markdown 渲染的正确性？

**难度：★★★**

**考点：**流式渲染、用户体验优化

**答案要点：**

打字机效果是通过增量更新 DOM 实现的，但难点在于处理不完整的 Markdown 语法标签。

- 增量渲染：前端接收到 SSE 的 chunk 后，拼接到已有的 content 字符串中。
- 语法截断处理：使用支持流式解析的 Markdown 库（如 markdown-it 或 micromark），或者在渲染前进行简单的闭合补全。
- 性能优化：不要每收到一个字符就触发一次全局 Re-render，可以利用 requestAnimationFrame 做节流渲染。
- 自动滚动：当用户处于底部时，新内容产生应触发自动吸底，但用户手动向上滚动时应停止。

**常见坑：**

- 渲染过程中代码块（Code Block）标签被截断导致样式错乱。
- 频繁更新 state 导致的性能瓶颈。

**追问：**

- 怎么判断用户当前是否处于“吸底”状态？

#### 4. 谈谈你对前端工程化中 Monorepo 架构的理解，以及在 AI 项目中的适用场景。

**难度：★★**

**考点：**工程化架构选型

**答案要点：**

Monorepo 是指在一个代码仓库中管理多个项目或模块的策略，常配合 pnpm workspaces 或 Turborepo 使用。

- 优势：代码复用方便、依赖版本统一、跨项目重构简单。
- AI 场景适用性：AI 产品通常包含 Web 端、插件端、管理后台、公共 UI 组件库和 AI SDK，非常适合 Monorepo。
- 挑战：CI/CD 耗时增加、权限管理困难、工具链配置复杂。
- 常用工具：pnpm (速度快、磁盘占用低)、Nx (功能全)、Turborepo (配置简单)。

**常见坑：**

- 依赖幽灵（Ghost Dependencies）问题。
- 仓库体积过大导致 Git 操作缓慢。

**追问：**

- 如何在 Monorepo 中实现只对修改过的包进行增量构建？

#### 5. 你们是如何监控 AI 产品的性能和异常的？特别是针对大模型响应慢的问题。

**难度：★★★**

**考点：**前端监控、性能度量

**答案要点：**

除了传统的 FP、LCP 指标，AI 产品更关注模型交互特有的指标。

- 核心指标：TTFT（首个 Token 返回时间）、Tokens Per Second（生成速度）、请求成功率。
- 异常捕获：SSE 连接中断、模型内容审核拦截（Sensitive Content）、解析 JSON 失败。
- 链路追踪：通过 TraceID 关联前端请求与后端模型调用日志。
- 用户反馈：收集点赞/点踩数据，作为模型质量评价的参考。

**常见坑：**

- 只监控 HTTP 状态码，忽略了业务逻辑上的“回复中断”。
- 忽略了长文本渲染导致的客户端卡顿监控。

**追问：**



- 如何量化 AI 回复的“首屏加载时间”？

## 6. 如何防御针对 AI 前端产品的 Prompt Injection（提示词注入）攻击？

难度：★★★

考点：安全、AI 交互安全

答案要点：

提示词注入是指用户通过特定的输入绕过系统设定的角色限制，前端需要配合后端进行多层防护。

- 输入过滤：在前端对敏感词、特殊指令字符（如 "Ignore previous instructions"）进行预检测。
- 长度限制：严格限制用户输入的长度，减少复杂注入脚本的可能。
- 角色隔离：在发送给后端的 Payload 中，明确区分 System Prompt 和 User Prompt。
- 输出转义：对 AI 返回的内容进行严格的 XSS 过滤，防止模型被诱导生成恶意脚本。

常见坑：

- 认为只要后端做了过滤，前端就可以完全信任 AI 返回的内容。

追问：

- 如果 AI 返回了一段 HTML 代码，你该如何安全地渲染它？

## 7. 介绍一下 Web Worker 的使用场景，它在 AI 前端处理中能起到什么作用？

难度：★★

考点：多线程编程

答案要点：

Web Worker 为前端提供了多线程能力，允许在后台线程执行耗时任务而不阻塞 UI。

- AI 场景应用：在前端进行大规模的向量计算（Vector Calculation）、本地模型推理（WebLLM）、或者对长文本进行复杂的正则匹配。
- 通信机制：通过 postMessage 传递数据，注意数据传输的拷贝开销（可使用 Transferable Objects 优化）。
- 限制：无法直接访问 DOM、window 对象和某些 Web API。
- 库支持：Comlink 等库可以简化 Worker 的调用方式。

常见坑：

- 在 Worker 中频繁进行大量数据的序列化/反序列化导致性能下降。

追问：

- SharedWorker 和普通 Worker 有什么区别？

## 8. 你们项目里是如何做 Webpack 或 Vite 性能优化的？

难度：★★

考点：构建工具优化

答案要点：

优化目标通常分为构建速度优化和产物性能优化两个维度。

- 构建速度：利用持久化缓存（filesystem cache）、多进程并行（thread-loader）、排除不必要的模块（exclude）。
- 产物性能：代码分割（Code Splitting）、Tree Shaking、压缩图片、Gzip/Brotli 压缩。
- Vite 特色：基于浏览器原生 ESM，开发环境秒级启动；生产环境使用 Rollup。
- AI 插件场景：对于一些巨大的 AI 相关库（如 transformers.js），采用 CDN 引入或异步加载。

**常见坑：**

- 盲目引入各种 Loader 导致构建反而变慢。
- 配置了 Tree Shaking 但因为副作用（Side Effects）没生效。

**追问：**

- 什么是模块联邦（Module Federation），它解决了什么问题？

## 三面（架构与综合能力）

### 1. 如果让你从零设计一个 AI Agent 的前端交互框架，你会考虑哪些核心模块？

**难度：★★★**

**考点：系统架构设计**

**答案要点：**

设计 AI Agent 框架需要超越简单的对话框，重点在于状态管理和多模态交互。

- 消息总线：处理流式输入、多轮对话上下文管理。
- 插件/工具系统：定义 Agent 如何调用前端能力（如绘图、搜索、修改 UI）。
- 渲染引擎：支持 Markdown、代码沙箱、图表、甚至动态 UI 组件。
- 状态机：管理 Agent 的不同阶段（思考中、执行中、已完成、错误）。
- 交互层：支持语音输入、文件拖拽、多模态内容预览。

**常见坑：**

- 状态管理过于耦合，导致难以支持“撤回”或“重新生成”功能。
- 忽略了 Agent 执行长任务时的中间状态展示。

**追问：**

- 如何实现 Agent 实时修改前端页面样式的预览功能？

### 2. 谈谈 RAG（检索增强生成）在前端链路中可以做哪些优化？

**难度：★★★**

**考点：RAG 流程理解与前端优化**

**答案要点：**



前端不仅是展示层，在 RAG 流程中也承担着数据预处理和引用展示的重要角色。

- 文档预处理：在上传文档时，前端可以先进行简单的清洗、分段预览。
- 引用溯源：AI 回复时，前端需将引用标注（Citations）与原始文档精确关联，实现点击跳转和高亮。
- 本地缓存：对频繁检索的 Embedding 或结果进行 IndexedDB 存储，减少网络开销。
- 交互反馈：允许用户对检索到的知识块进行“相关性”评价，反哺后端 RAG 效果。

**常见坑：**

- 引用标注展示不清晰，用户无法判断 AI 是否在“胡说八道”。

**追问：**

- 如果 PDF 文档非常大，前端如何实现高效的局部预览和高亮？

### 3. 在 AI 产品快速迭代的过程中，如何平衡“业务交付速度”与“代码质量”？

**难度：★★★**

**考点：**项目管理与技术决策

**答案要点：**

这是一个权衡艺术，核心在于建立可扩展的基础设施和合理的规范。

- 核心组件标准化：沉淀一套 AI 专用的 UI 组件库（对话框、加载动画、Token 计数器）。
- 逻辑解耦：将模型调用逻辑与业务 UI 分离，方便未来更换模型供应商。
- 自动化测试：优先覆盖核心交互链路（如流式解析、鉴权），而非追求 100% 覆盖率。
- 技术债管理：通过 TODO 标记和定期重构，在业务间隙偿还快速迭代留下的坑。

**常见坑：**

- 为了快直接在页面组件里写死 Prompt，导致后期难以维护。

**追问：**

- 当产品经理提出一个可能导致严重性能问题的 AI 交互需求时，你会如何沟通？

### 4. 你认为未来 1-2 年，前端开发在 AI 浪潮下会发生哪些本质的变化？

**难度：★★★**

**考点：**技术视野与前瞻性

**答案要点：**

前端将从单纯的“界面开发”向“交互智能体开发”转型。

- 开发模式：从手写代码转向 Copilot 辅助编程，甚至自然语言生成 UI（Text-to-UI）。
- 交互形态：从传统的点击/滑动转向 LUI（语言交互界面）和 GUI 结合的混合模式。
- 边缘计算：更多的小型模型将直接在浏览器运行（WebGPU/WASM），实现隐私保护和零延迟。
- 角色转变：前端需要懂一些模型参数调优（Temperature、TopP）和 Prompt 工程。

**常见坑：**

- 过于悲观认为前端会消失，或者过于乐观认为不需要学习新技术。

**追问：**

- 你最近在关注哪些 AI 相关的开源项目或技术趋势？

## 5. 聊聊你在过去一年中遇到的最具有挑战性的技术问题，你是如何解决的？

**难度：★★★**

**考点：**问题解决能力、复盘深度

**答案要点：**

通过具体的案例展示你的思考深度和技术攻坚能力。

- 背景：清晰描述问题发生的场景和影响范围。
- 分析：展示你如何定位问题（抓包、性能分析工具、源码阅读）。
- 方案对比：为什么选择了 A 方案而不是 B 方案，做了哪些权衡。
- 结果与沉淀：问题解决后的效果，以及是否形成了通用的解决方案或文档。

**常见坑：**

- 描述过于流水账，没有突出技术难点。
- 解决方案过于平庸，体现不出资深开发者的水平。

**追问：**

- 如果现在让你重新做一遍，你会尝试什么不同的方法？